SPATIAL ECOLOGY

Neural Nets & Convolutional Neural Networks

Raffaele Scaringi

Agenda

1) Preparing got the Paper discussion (Nov. 12);
2) Feedforward Neural Networks

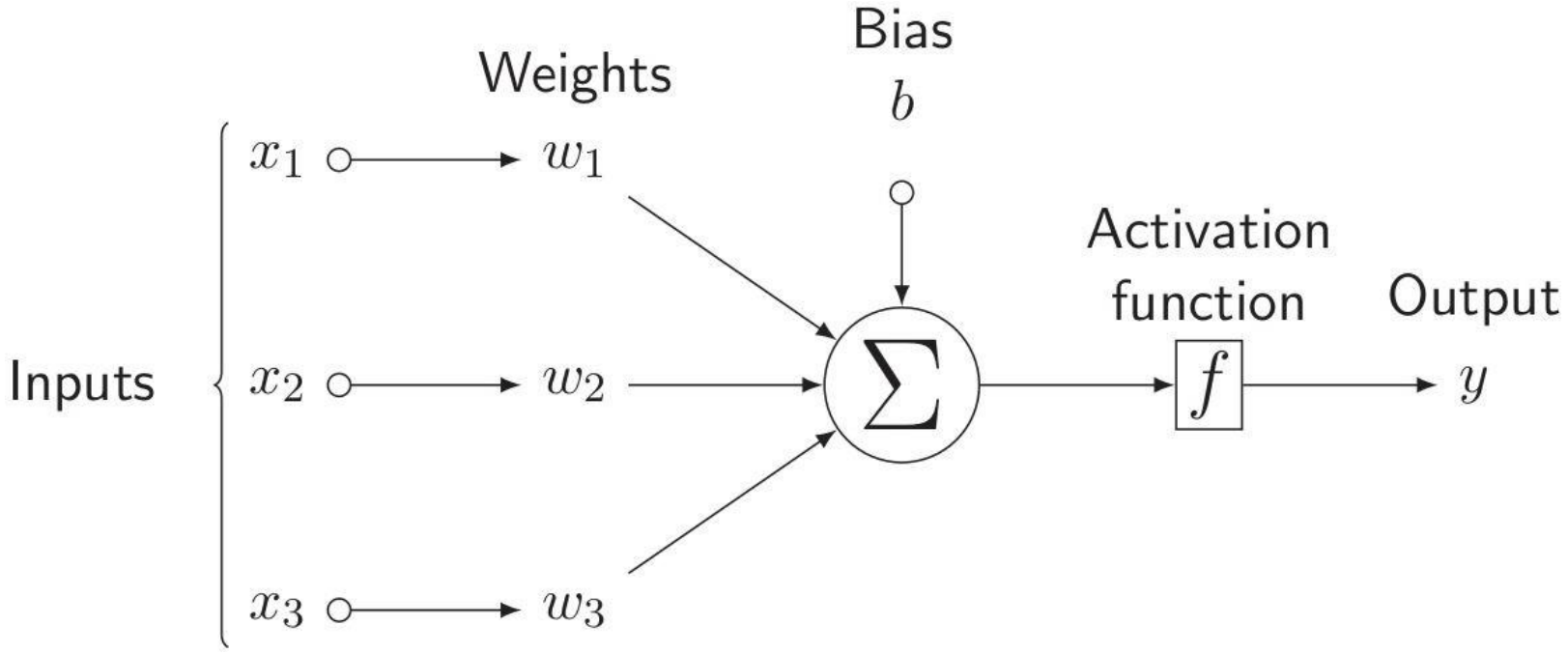
- The limitations of Perceptrons
- Multi-layer Perceptron
- Training: the forward and back-propagation
- Debugging tips
- Tutorial: Neural Nets for the tree height dataset

2) Convolutional Neural Networks

- Spatial locality structure
- Kernels, padding, pooling
- Classification tasks
- Saliency Analysis
- Tutorial: data batching, classification of satellite images, WandB

Perceptron: Threshold Logic

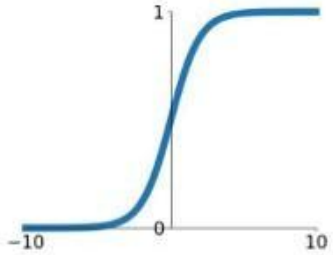
$$\mathcal{L}_{\text{perc}}(\mathbf{x}, y) = \begin{cases} 0 & \text{if } y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) > 0 \\ -y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) & \text{if } y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) \leq 0 \end{cases}$$



Activation functions

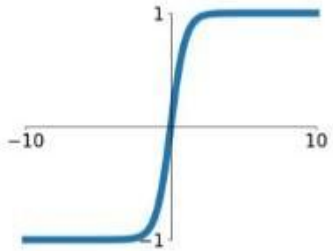
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



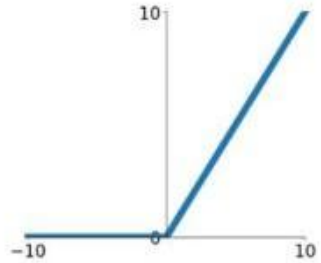
tanh

$$\tanh(x)$$



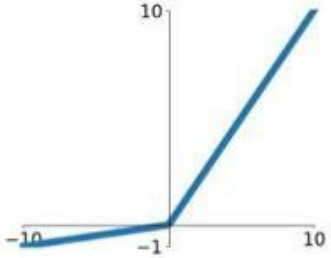
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

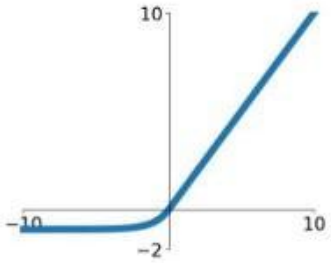


Maxout

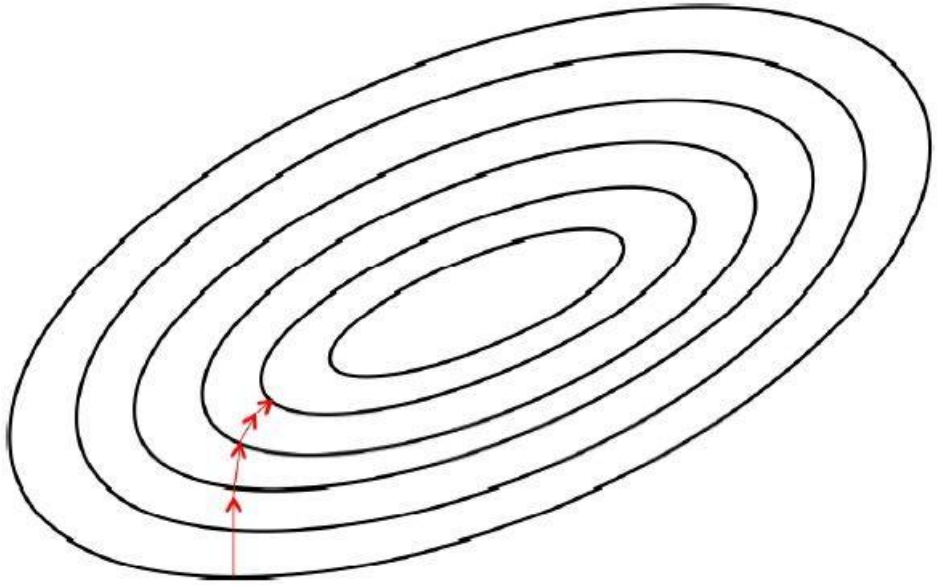
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Optimizers (pt1)

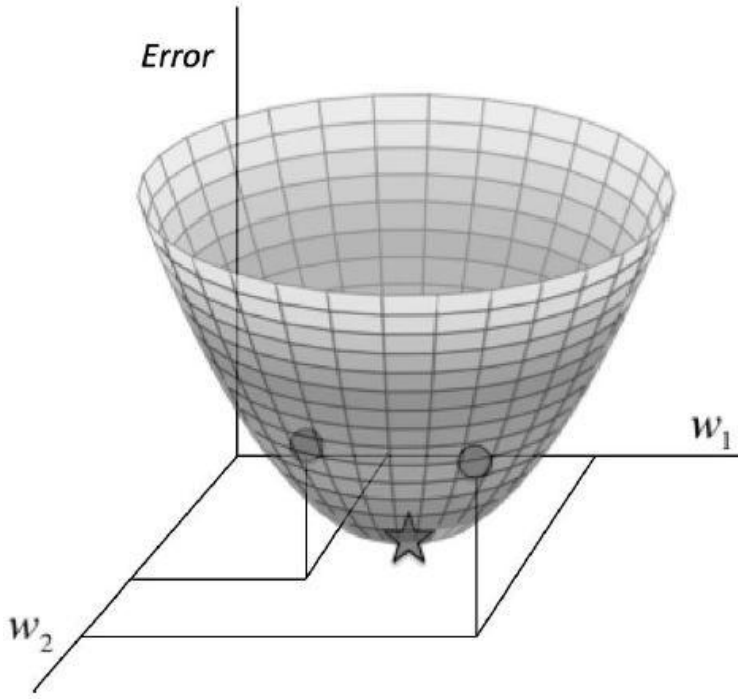


Gradient

$$\Delta w_k = -\frac{\partial E}{\partial w_k}$$

$$= -\frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + \Delta w_k$$



Stochastic gradient descent (**SGD**)

Optimizers



Watch out for local minimal areas

Hyperparameters

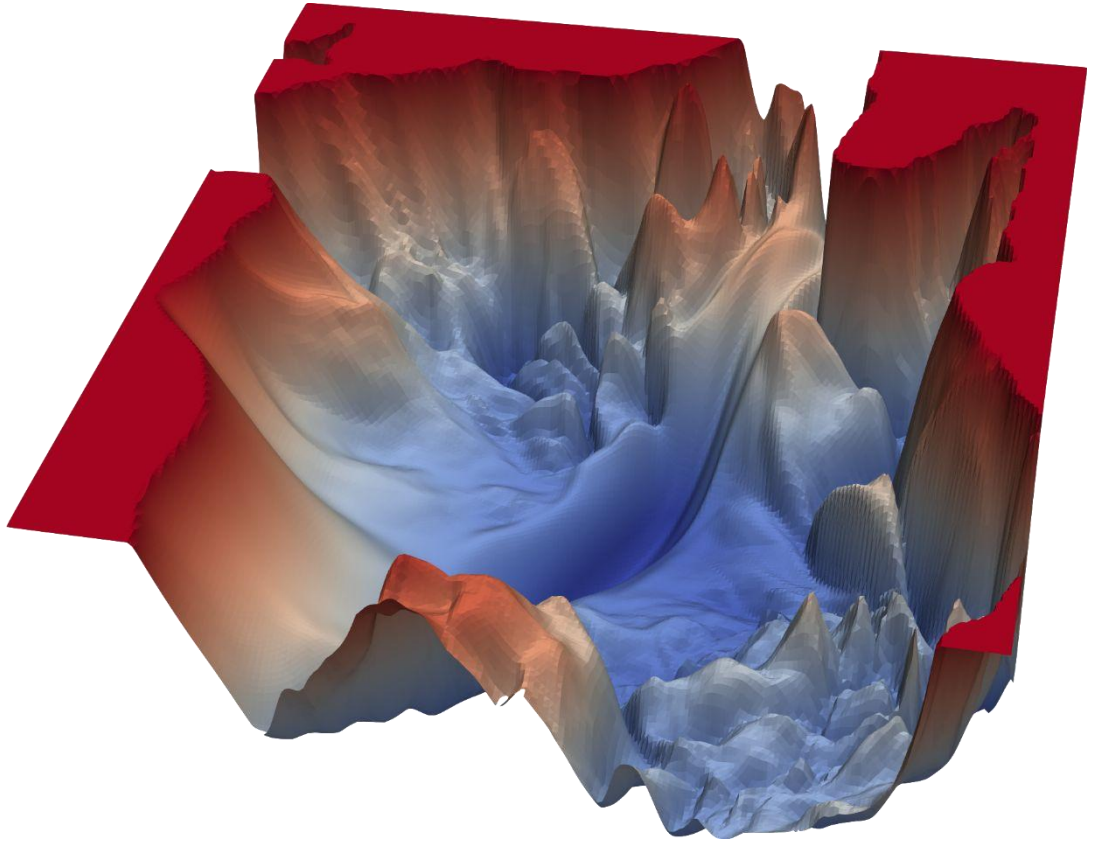
- Learning rate (α)

$$\Delta w_k = -\alpha \frac{\partial E}{\partial w_k}$$

$$= -\alpha \frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)



Optimizers

Optimizers

Hyperparameters

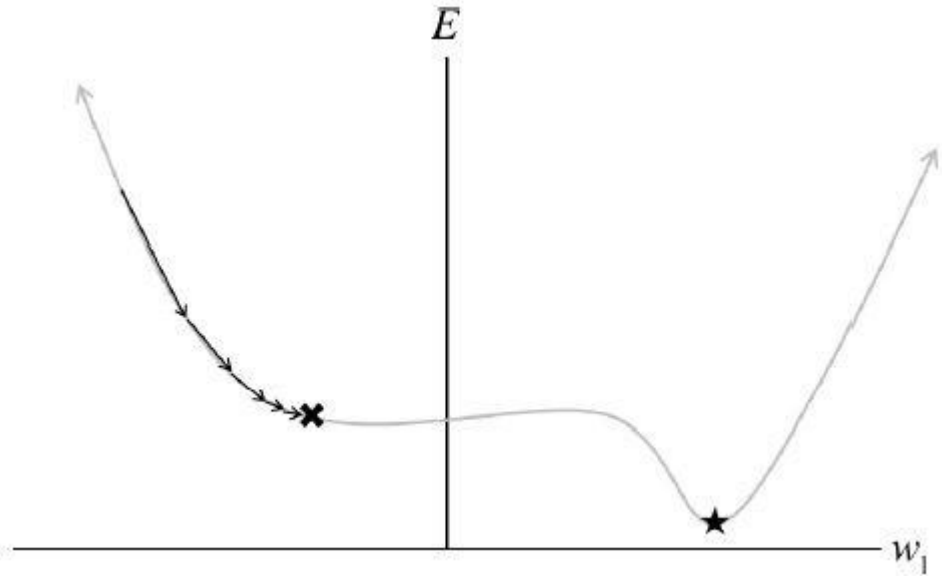
- Learning rate (α)

$$\Delta w_k = -\alpha \frac{\partial E}{\partial w_k}$$

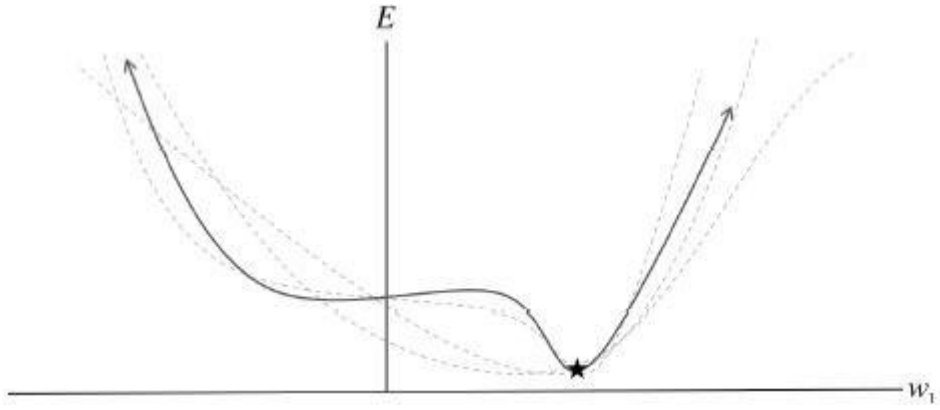
$$= -\alpha \frac{\partial}{\partial w_k} \left(\frac{1}{m} \sum_i (w^T X_i - y_i)_i^2 \right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)



Local Minima



Multiple samples

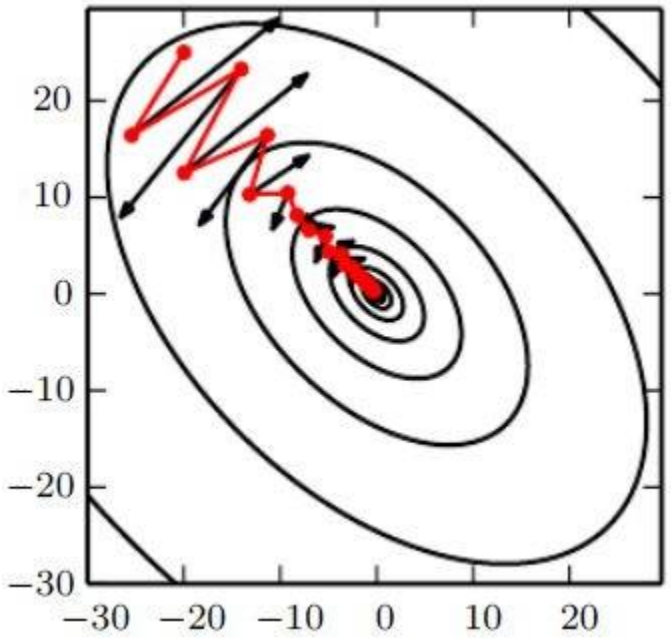
Optimizers

Hyperparameters

- Learning rate (α)
- Momentum (β)

$$v_{i+1} = v\beta - \alpha \frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + v$$



SGD

SGD+Momentum

Stochastic gradient descent with momentum (**SGD+Momentum**)

Optimizers

Hard to pick right hyperparameters

- Small learning rate: long convergence time
- Large learning rate: convergence problems

Adagrad: adapts learning rate to each parameter

$$\Delta w_{k,t} = -\alpha \frac{\partial E_t}{\partial w_{k,t}} = -\alpha \nabla_w E(w_t)$$

- Learning rate might decrease too fast
- Might not converge

$$g_{t,i} = \nabla_w E(w_{t,i})$$

$$G_{t+1,i} = G_{t,i} + g_{t,i} \odot g_{t,i}$$



$$w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{G_{t,i} + \epsilon}} g_{t,i}$$

Optimizers

RMSprop: decaying average of the past squared gradients

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2$$

Exponentially decaying average

Adadelta

$$E[\Delta_w^2]_t = \gamma E[\Delta_w^2]_{t-1} + (1-\gamma)\Delta_w^2$$

$$\Delta w_t = \frac{\sqrt{E[\Delta_w^2]_t + \epsilon}}{\sqrt{G_{t,i} + \epsilon}} g_t$$

$$\Delta w_{k,t} = -\alpha \frac{\partial E_t}{\partial w_{k,t}} = -\alpha \nabla_w E(w_t) = -\alpha g_{t,i}$$

$$g_{t,i} = \nabla_w E(w_{t,i})$$

$$G_{t+1,i} = \gamma G_{t,i} + (1-\gamma) g_{t,i} \odot g_{t,i}$$



$$w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{G_{t,i} + \epsilon}} g_{t,i}$$

Optimizers

ADAM: decaying average of the past gradients and its square

RMSprop / Adadelta

$$g_{t,i} = \nabla_w E(w_{t,i})$$

$$G_{t+1,i} = \gamma G_{t,i} + (1-\gamma) g_{t,i} \odot g_{t,i}$$



$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

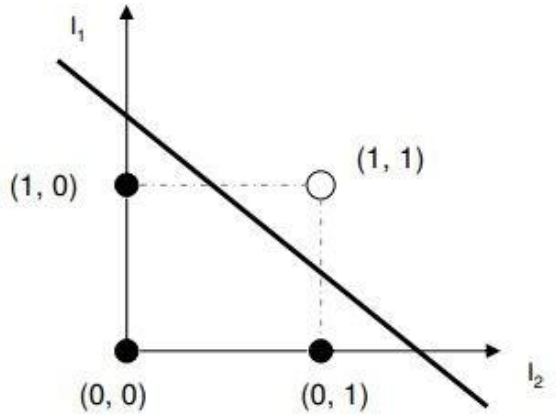
$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

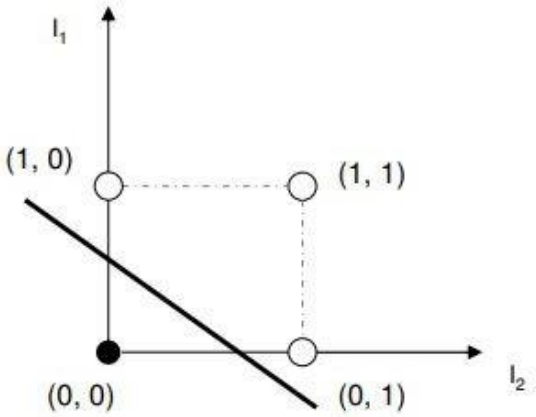
Multi-layer Perceptron

Limitations of the Perceptron

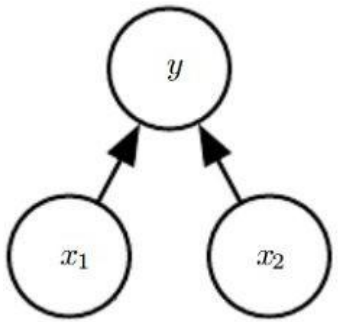
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

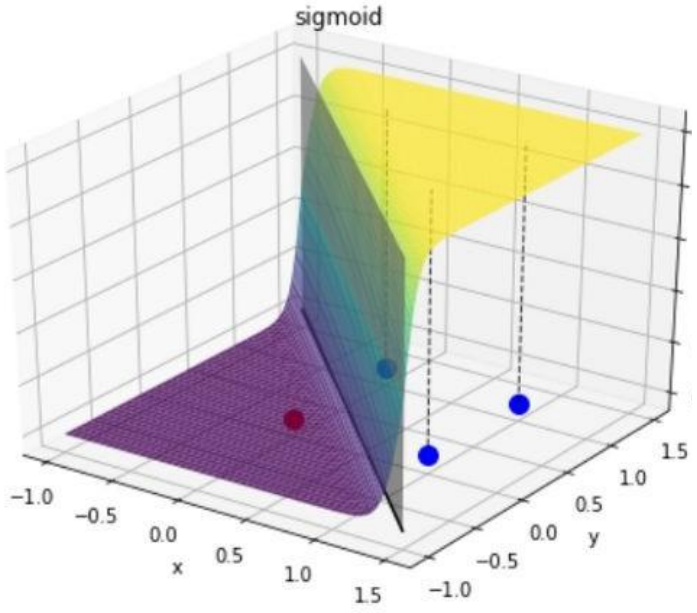


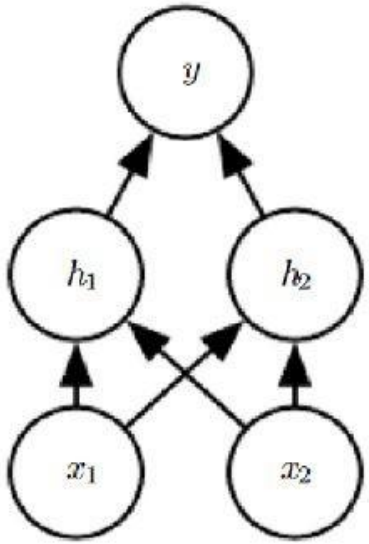
OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



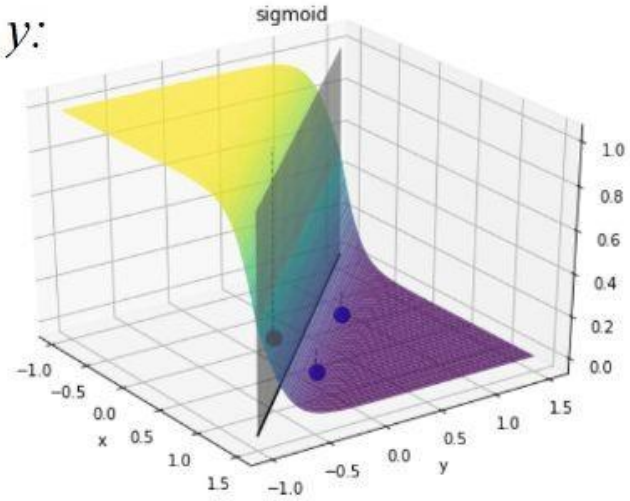
Perceptron

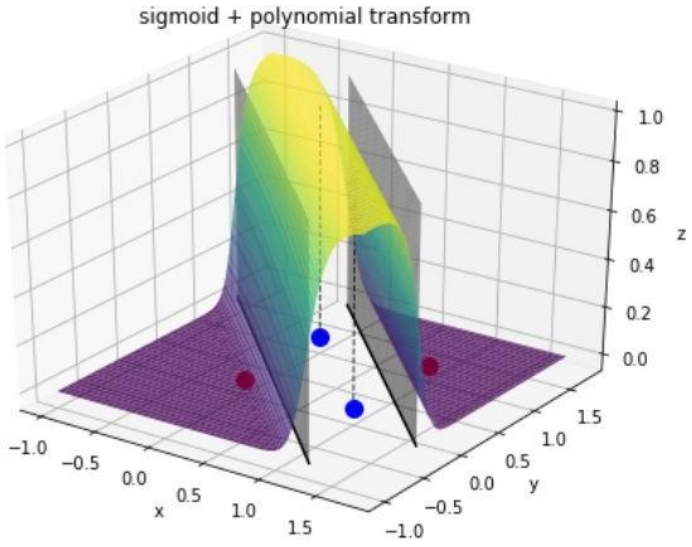

y
x_1
x_2


sigmoid
x
y

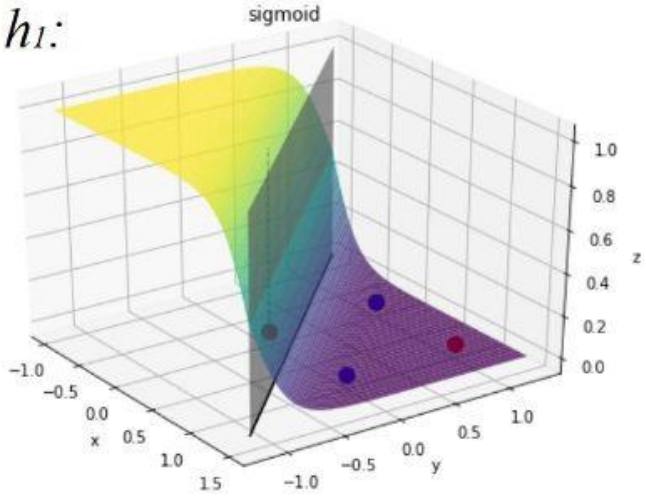

y
h_1
h_2
x_1
x_2

y:

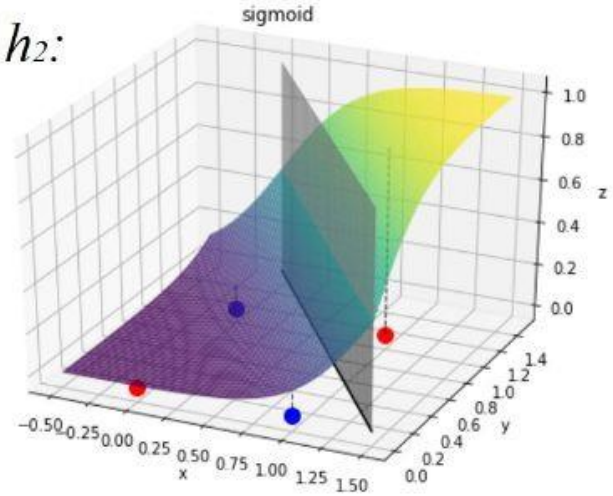

sigmoid
x
y


sigmoid + polynomial transform
x
y
z

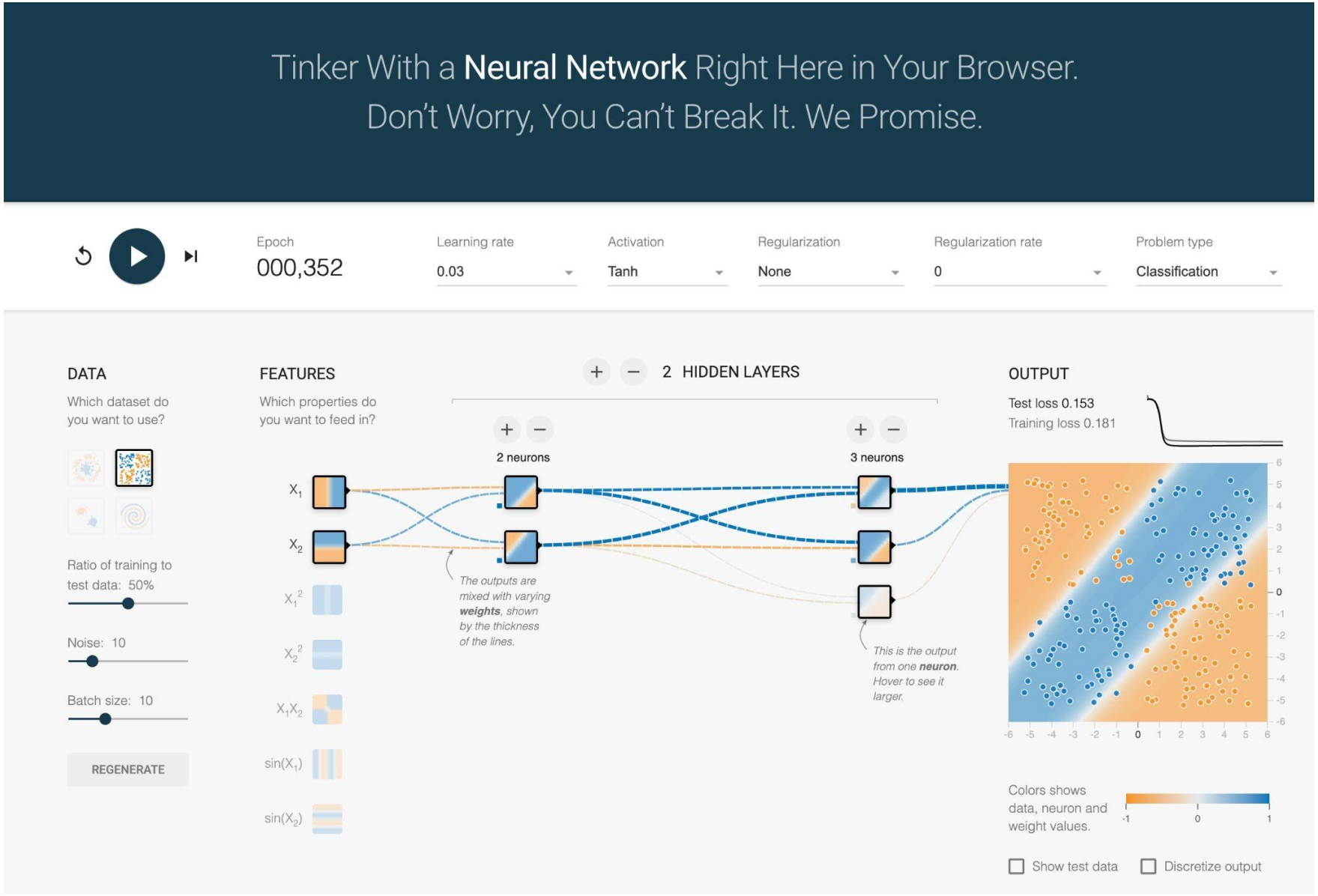
h1:


sigmoid
x
y
z

h2:

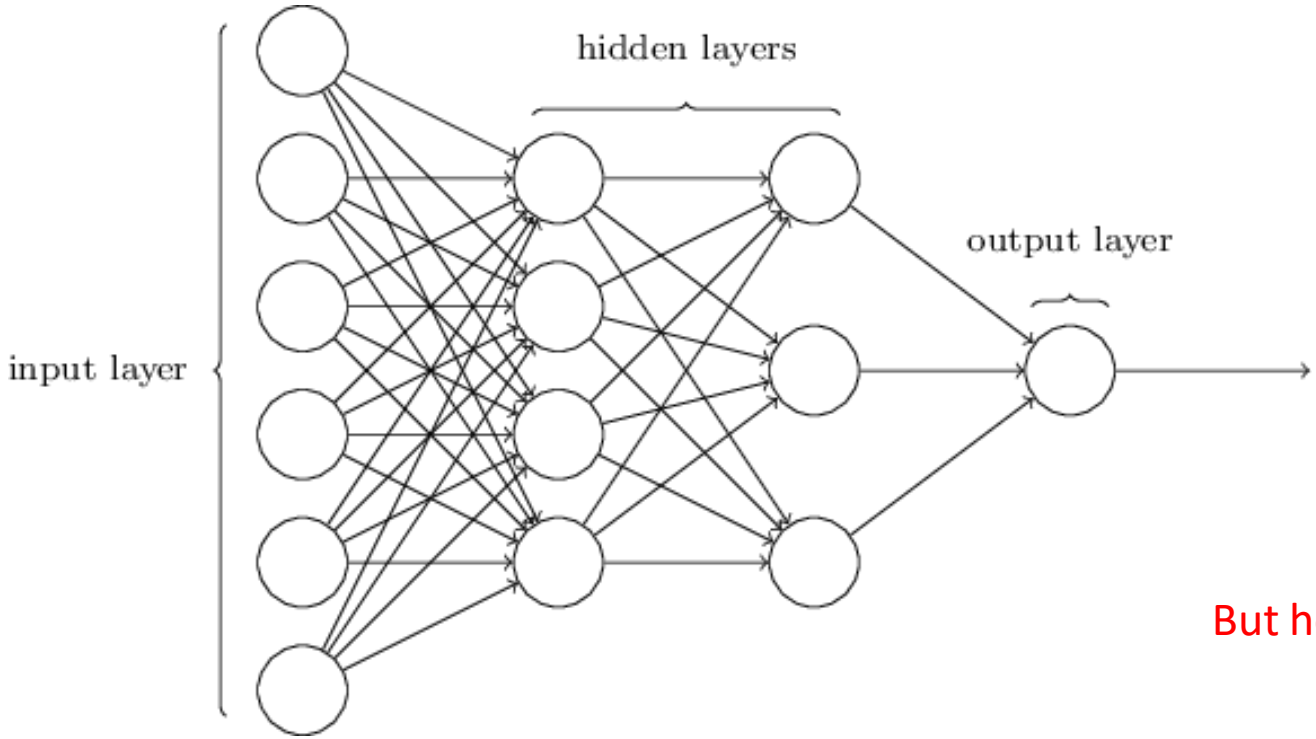

sigmoid
x
y
z

Let’s play with it!



Try it [here]

Architecture of Neural Networks



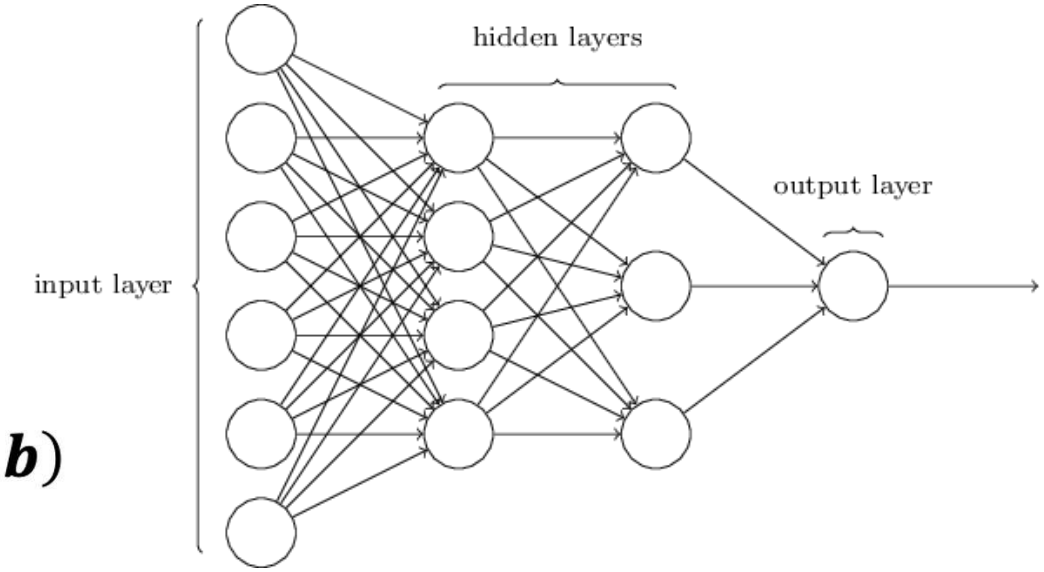
But how do we train it?

- Sometimes called multi-layer perceptron (MLP)
- Output from one layer is used as input for the next (Feedforward network)

Forward Propagation

- Store weights and biases as matrices
- Suppose we are considering the weights from the second (hidden) layer to the third (output) layer
  - w is the weight matrix with w_{ij} the weight for the connection between the ith neuron in the second layer and the jth neuron in the third layer
  - b is the vector of biases in the third layer
  - a is the vector of activations (output) of the 2nd layer
  - a' is the vector of activations (output) of the third layer

$$\boldsymbol{a' = \sigma(wa + b)}$$



Backpropagation

1. **Input** x**:** Set the corresponding activation a^1 for the input layer.

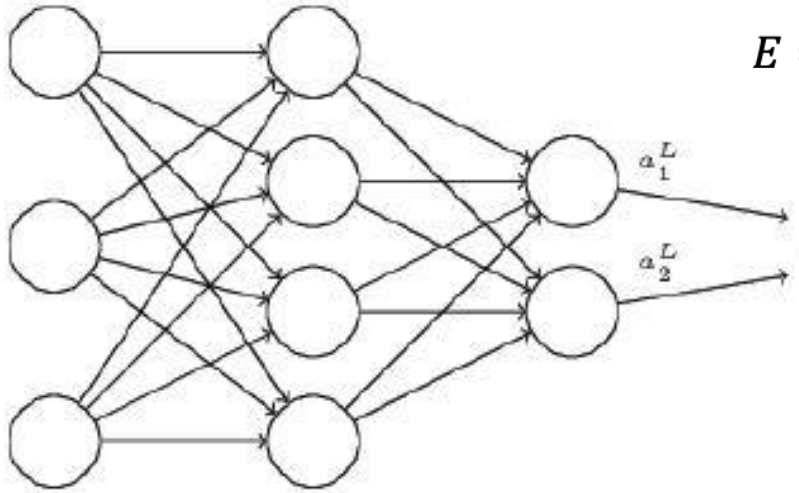
2. **Feedforward:** For each $l = 2, 3, \ldots, L$ compute $z^l = w^l a^{l-1} + b^l$ and $a^l = \sigma(z^l)$.

3. **Output error** δ^L**:** Compute the vector $\delta^L = \nabla_a C \odot \sigma'(z^L)$.

4. **Backpropagate the error:** For each $l = L-1, L-2, \ldots, 2$ compute $\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$.

5. **Output:** The gradient of the cost function is given by $\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \delta^l_j$ and $\frac{\partial C}{\partial b^l_j} = \delta^l_j$.

$$\frac{\partial E}{\partial w^l_{ji}} = \frac{\partial E}{\partial a^l_i}\frac{\partial a^l_i}{\partial z^l_j}\frac{\partial (w^l_{ji} a^{l-1}_i)}{\partial w^l_{ji}}$$



$$E = \frac{1}{2}\sum_i (a_i^L - y_i)^2$$

$$cost\ E = E(a^L)$$

$$z^l_j = \sum_i w^l_{ji} a^{l-1}_i + b^l_j \qquad a^l_j = \sigma\left(\sum_i w^l_{ji} a^{l-1}_i + b^l_j\right) = \sigma(z^l_j)$$

$$\delta^L_j \equiv \frac{\partial E}{\partial z^L_j} = \frac{\partial E}{\partial a^L_i}\frac{\partial a^L_i}{\partial z^L_j} = \frac{\partial E}{\partial a^L_j}\sigma'(z^L_j) \qquad (1)$$

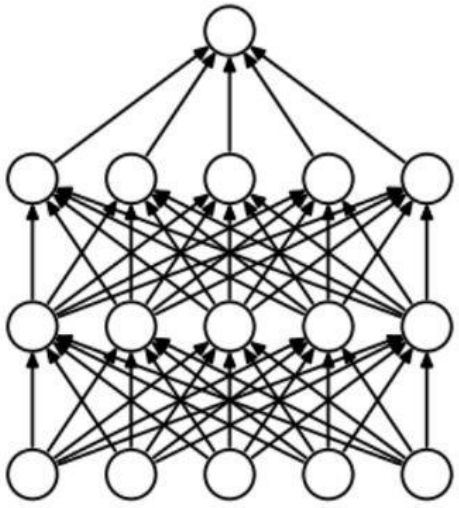
$$\delta^l_j \equiv \frac{\partial E}{\partial z^l_j} = \frac{\partial E}{\partial z^{l+1}_i}\frac{\partial z^{l+1}_i}{\partial z^l_j} = \frac{\partial z^{l+1}_i}{\partial z^l_j}\delta^{l+1}_i$$

$$= \frac{\partial (\sum_i w^{l+1}_{ij} a^l_j + b^{l+1}_i)}{\partial z^l_j}\delta^{l+1}_j = \sum_i w^{l+1}_{ij}\delta^{l+1}_i \sigma'(z^l_j) \qquad (2)$$

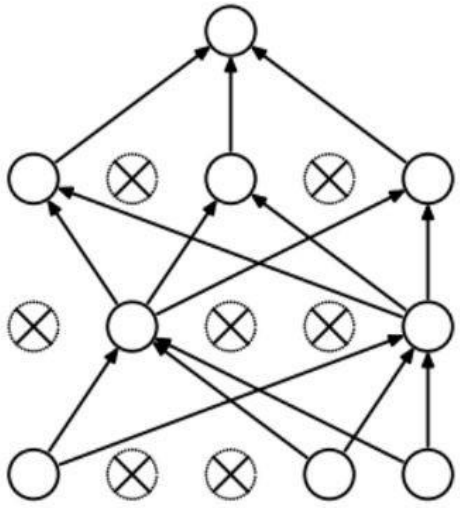
Extra Regularization for Neural Nets

Dropout: accuracy in the absence of certain information

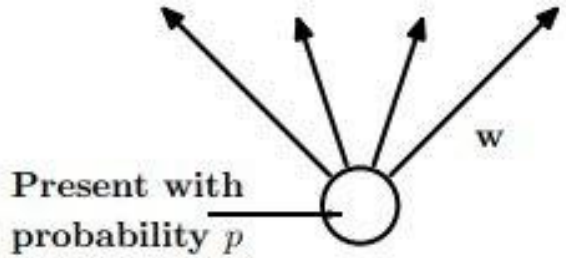
- Prevent dependence on any one (or any small combination) of neurons



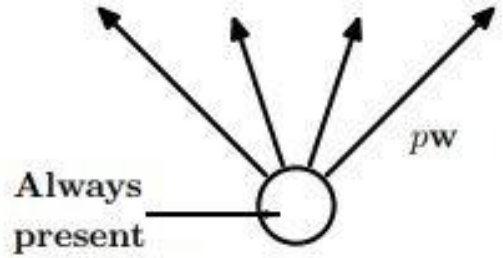
(a) Standard Neural Net



(b) After applying dropout.



(a) At training time



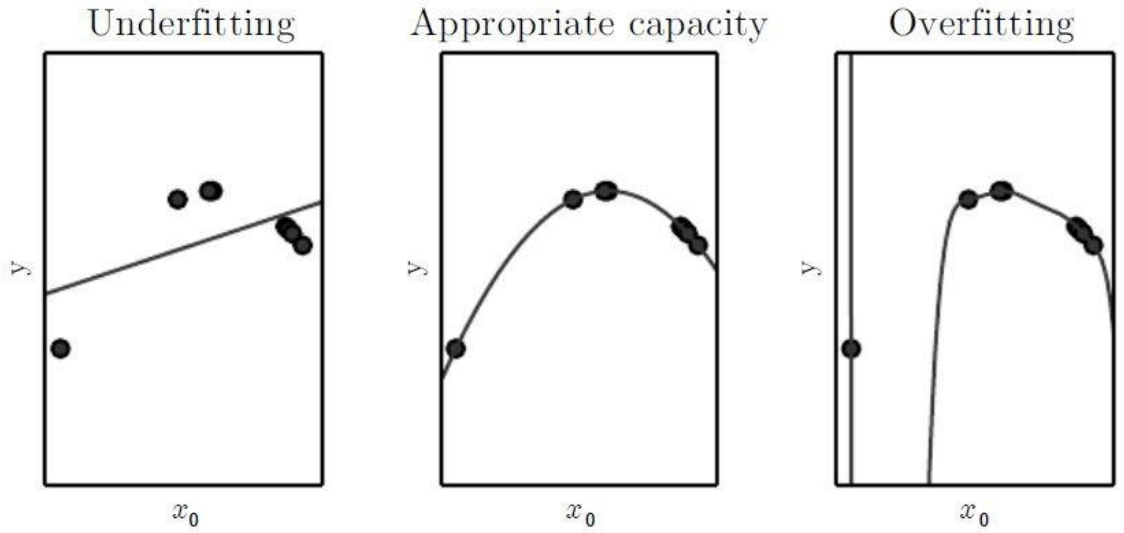
(b) At test time

Capacity, Overfitting and Underfitting

1) Make training error small
2) Make the gap between training and test error small

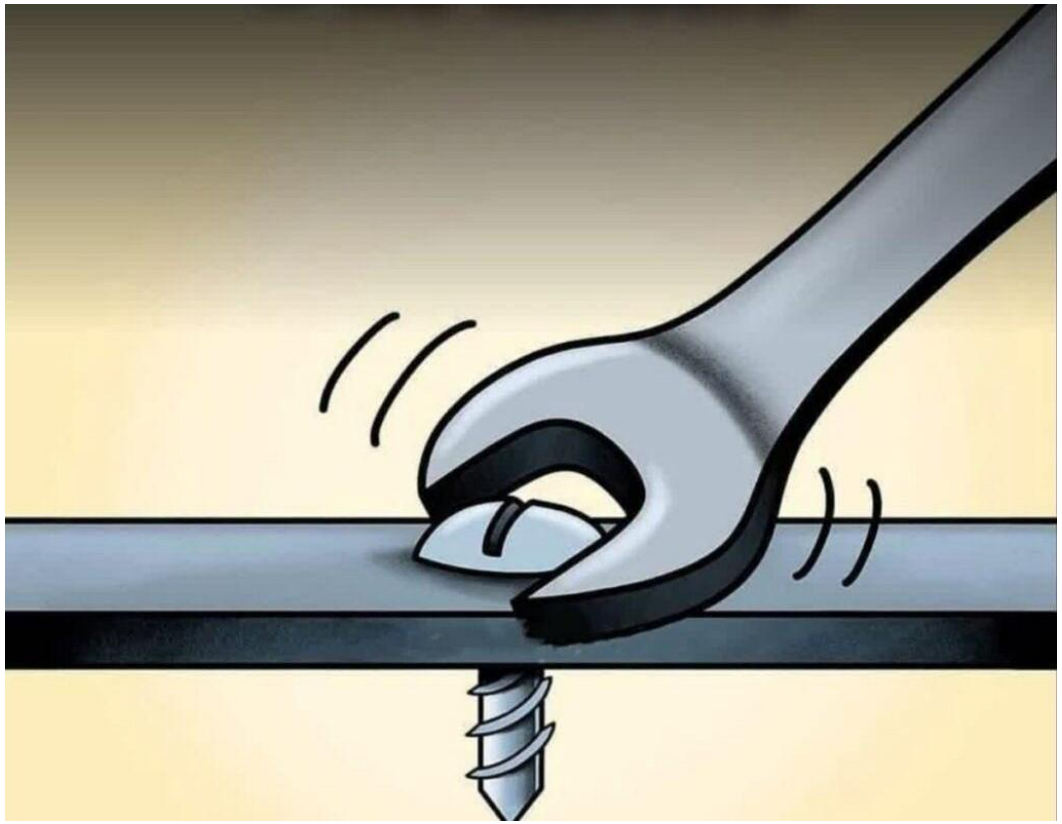
Capacity (fitting ability)

Low capacity ← $\hat{y} = \sum_{i}^{1} w_i x^i$ $\hat{y} = \sum_{i}^{2} w_i x^i$ $\hat{y} = \sum_{i}^{9} w_i x^i$ → High capacity



Back to the code

When people want to use Machine Learning without math



How training works

1. In each ***epoch***, randomly shuffle the training data
2. Partition the shuffled training data into ***mini-batches***
3. For each mini-batch, apply a single step of **gradient descent**
   - **Gradients** are calculated via ***backpropagation*** (the next topic)
4. Train for multiple epochs

Debugging a neural network

- What can we do?
  - Should we change the learning rate?
  - Should we initialize differently?
  - Do we need more training data?
  - Should we change the architecture?
  - Should we run for more epochs?
  - Are the features relevant for the problem?
- Debugging is an art
  - We'll develop good heuristics for choosing good architectures and hyper parameters

Extra readings

Deep Learning book:

- Chapter 5.9: Intro to Stochastic Gradient Descent (SGD)
- Chapter 6: Multilayer perceptrons
- Chapter 6.2.2: Output Units (Activation functions)
- Chapter 6.5: Back-Propagation
- Chapter 8.3: Basic Algorithms (Optimizers)

Convolutional Neural Networks

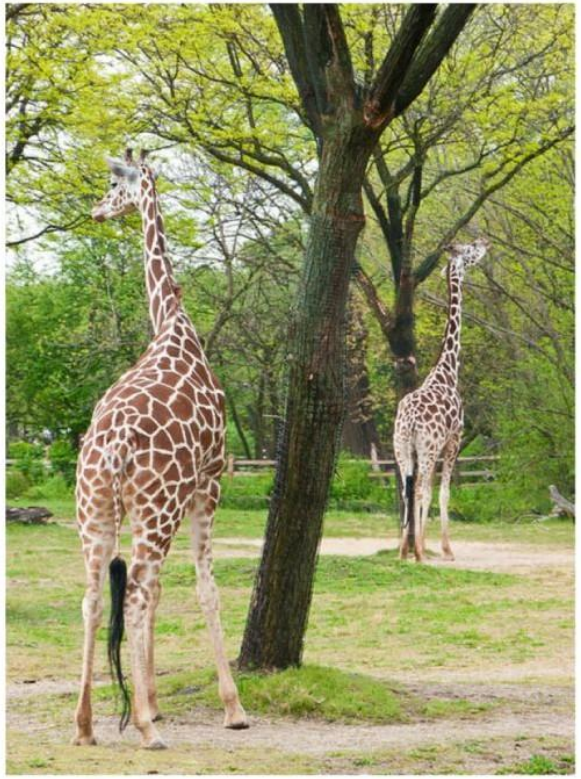
Taking An Image as an Input



a soccer player is kicking a soccer ball



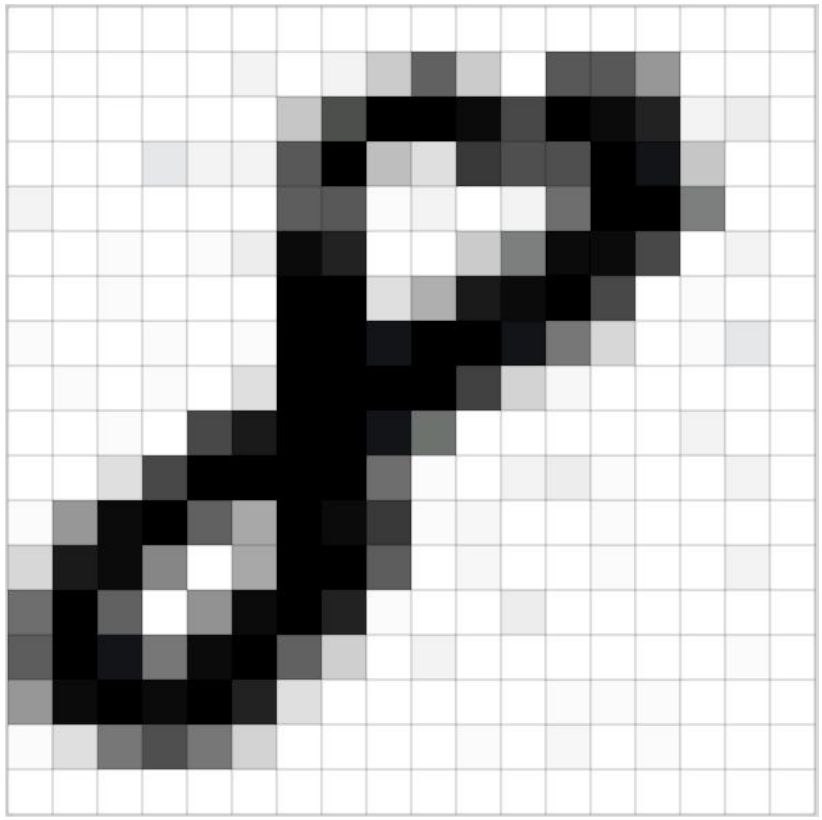
a street sign on a pole in front of a building


a couple of giraffe standing next to each other

NNs have been very good at classifying real world images

First architecture was LeNet formulated by Yann Lecun in 1988

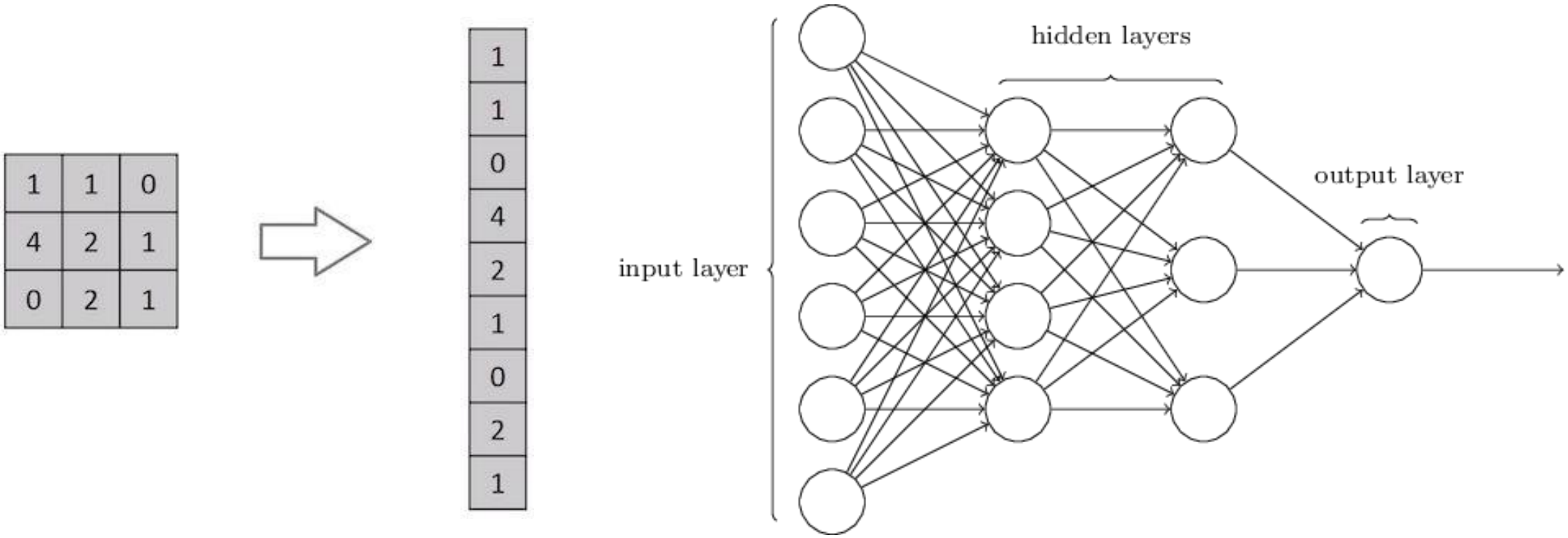
Images are a series of Pixel Values



Grayscale images:
0=Black
255 = White

Spatial locality structure

Handling images with Neural Networks



Works well for simple images, but fails when there are more complex patterns in the image

Local receptive fields

Make connections in small, localized regions of the input image

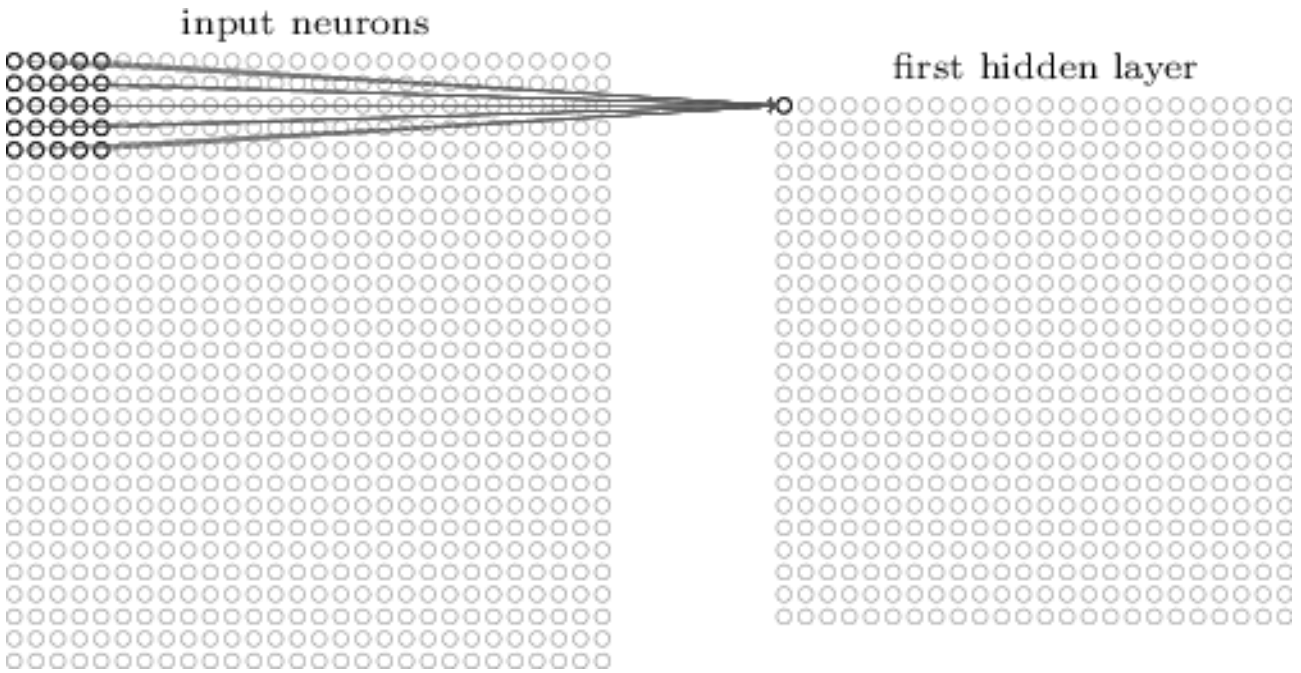


Image taken from Michael Neilsen's book "Neural Networks and Deep Learning"

Local receptive fields

Slide the local receptive field over by one (or more) pixel and repeat

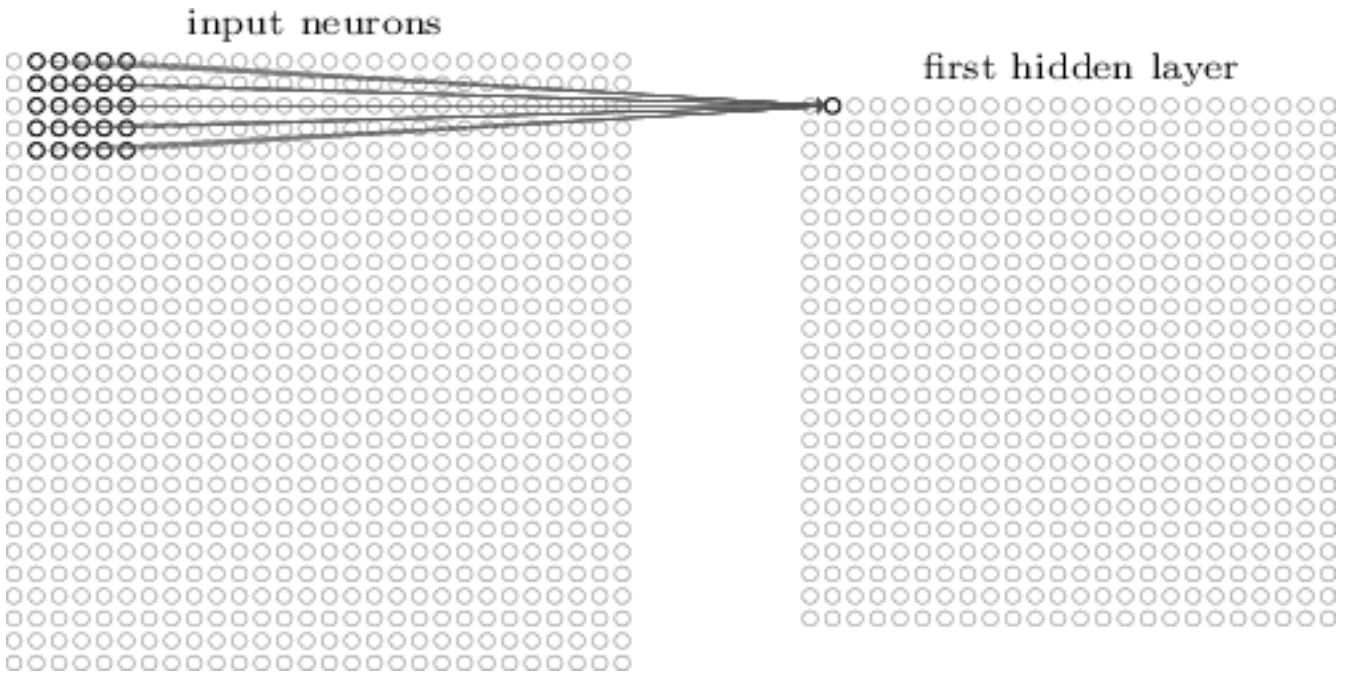


Image taken from Michael Neilsen's book "Neural Networks and Deep Learning"

The convolution operation

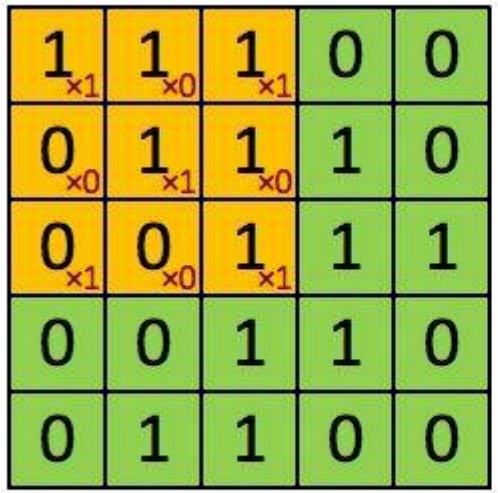
Image

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

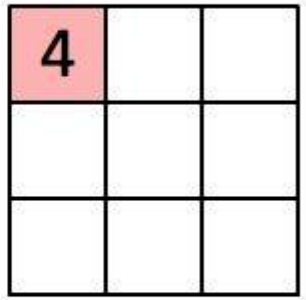
1	0	1
0	1	0
1	0	1

Filter/
Feature detector

1. Pointwise multiply
2. Add results
3. Translate filter

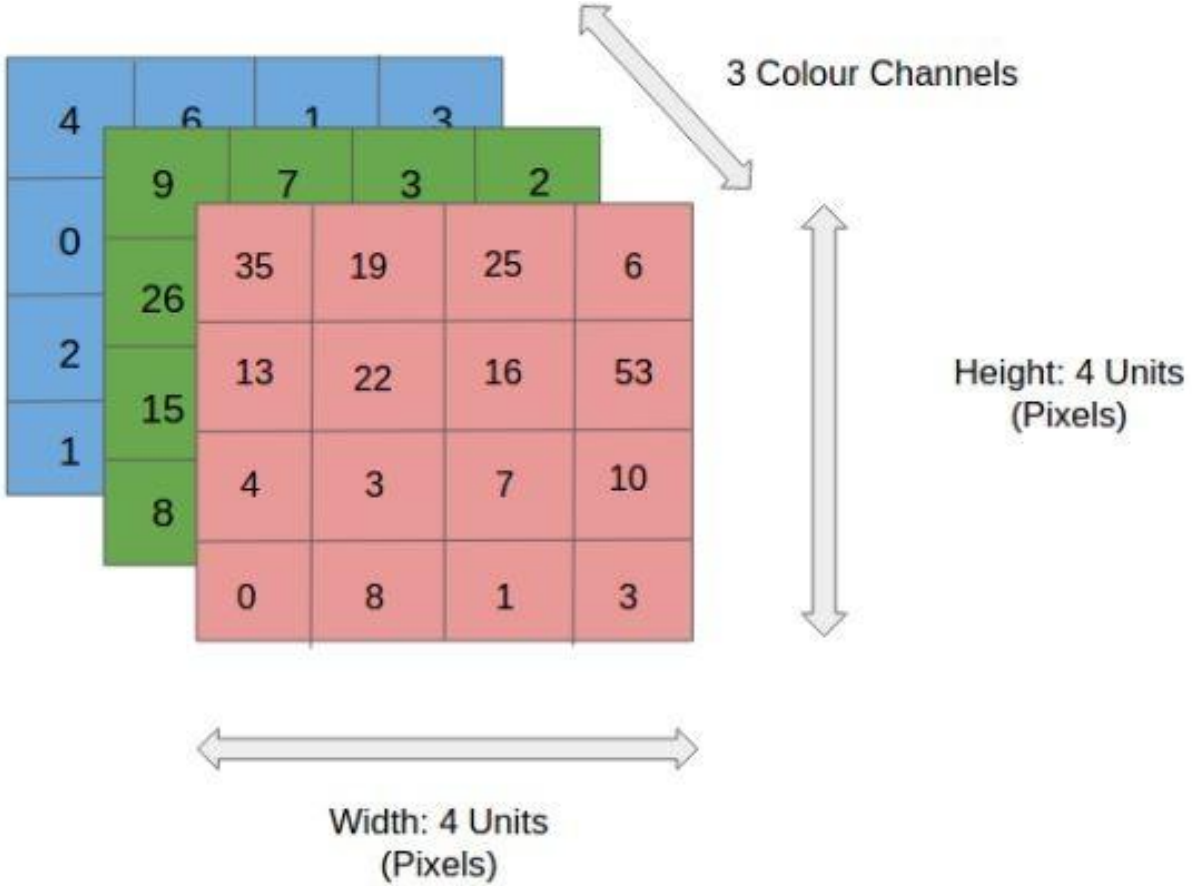


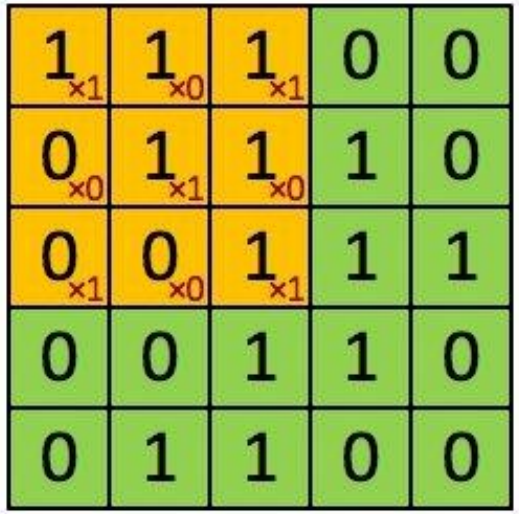
Image

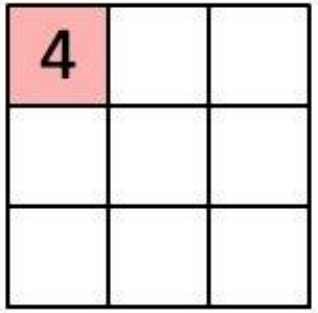


Convolved
Feature

Convolutional Neural Networks







Filters

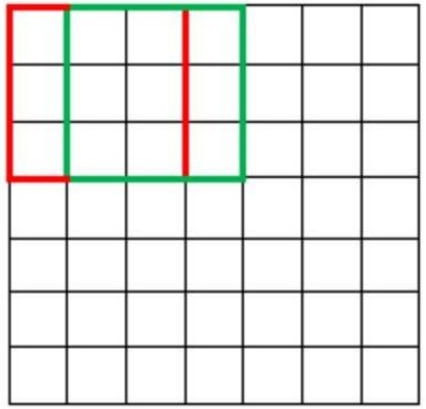
Original Image



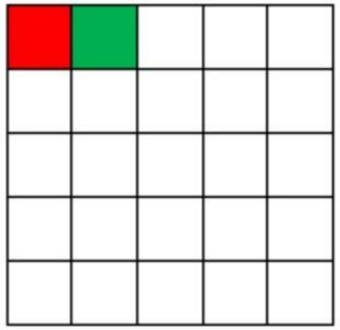
Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Stride

7 x 7 Input Volume

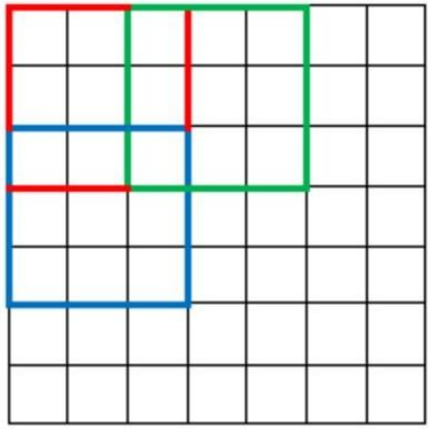


5 x 5 Output Volume

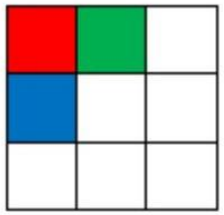


Stride 1

7 x 7 Input Volume



3 x 3 Output Volume



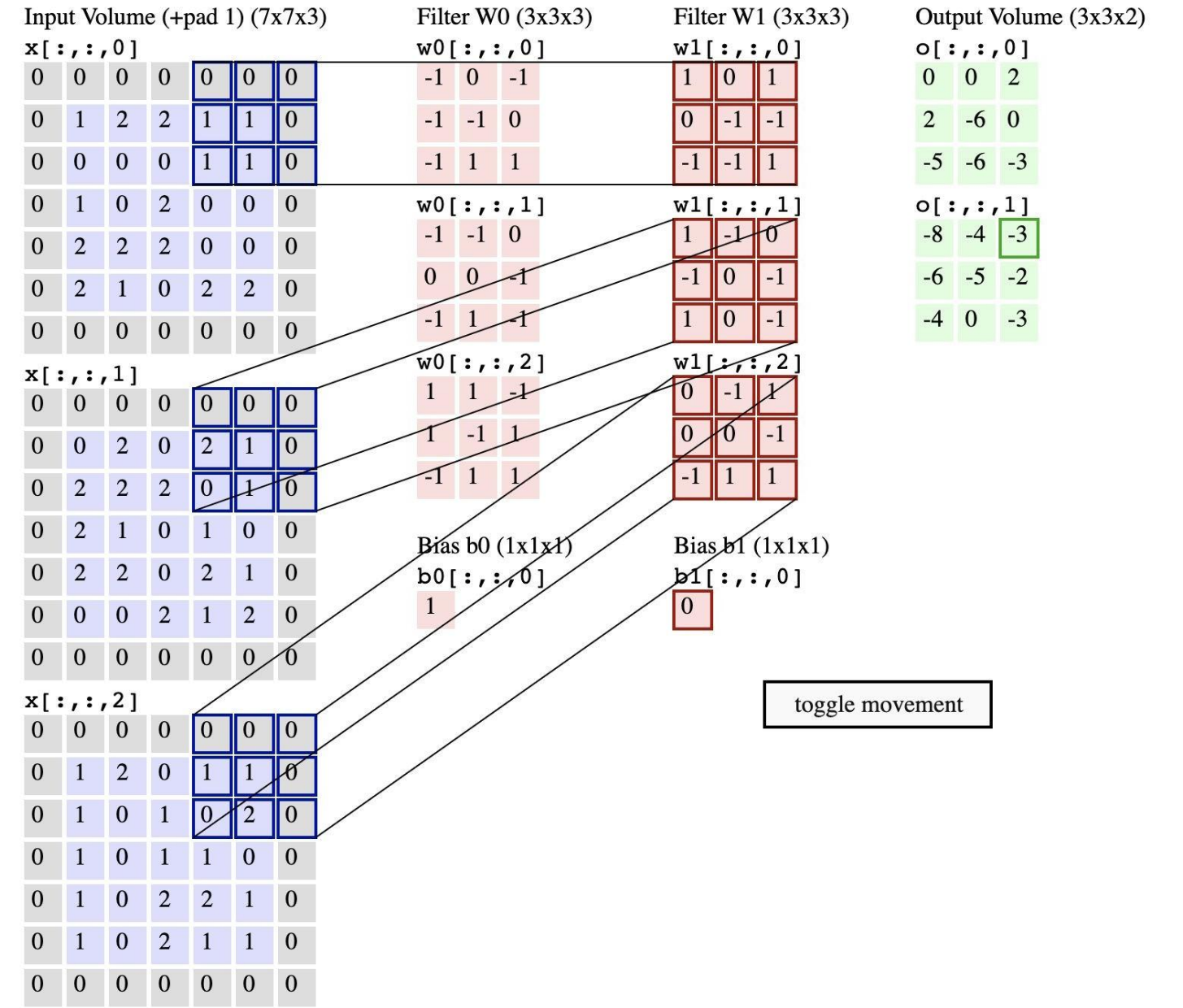
Stride 2

CNN over the image channels

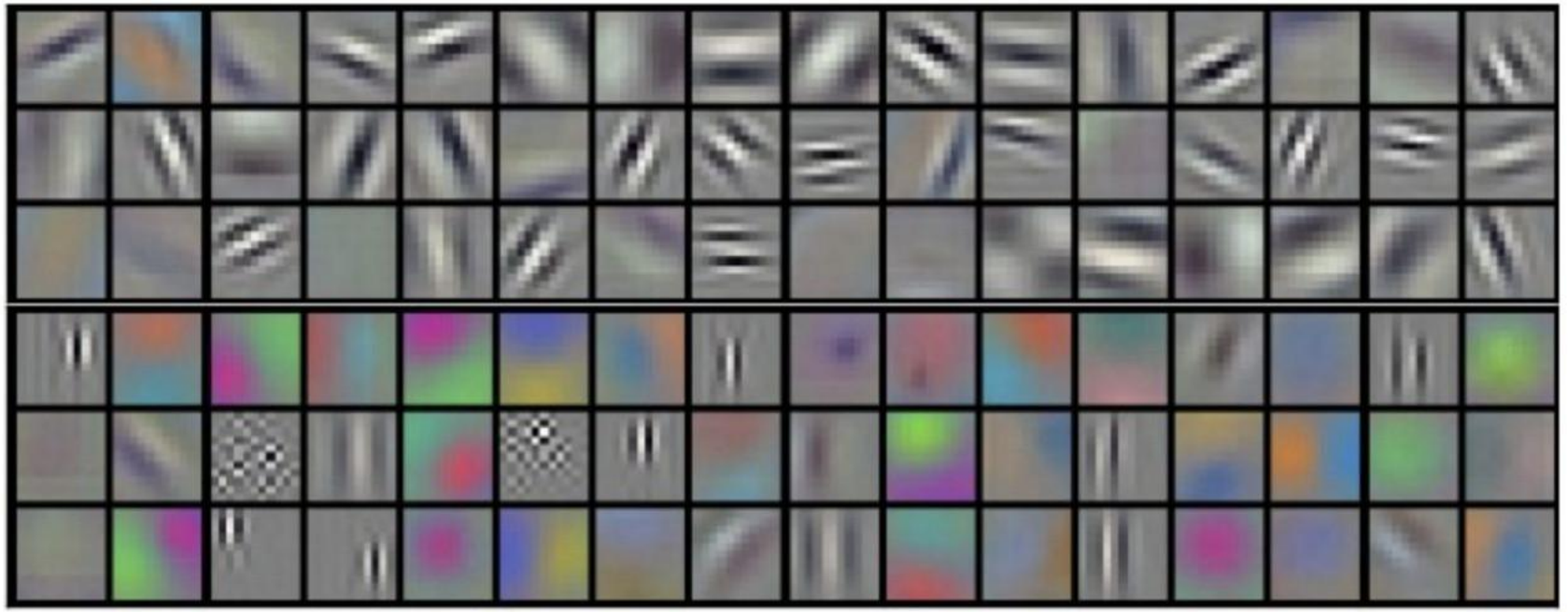
- Input: $W \times H \times D$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P
- Output: $W_2 \times H_2 \times D_2$

where:

- $W_2 = (W - F + 2P)/S + 1$
- $H_2 = (H - F + 2P)/S + 1$
- $D_2 = K$

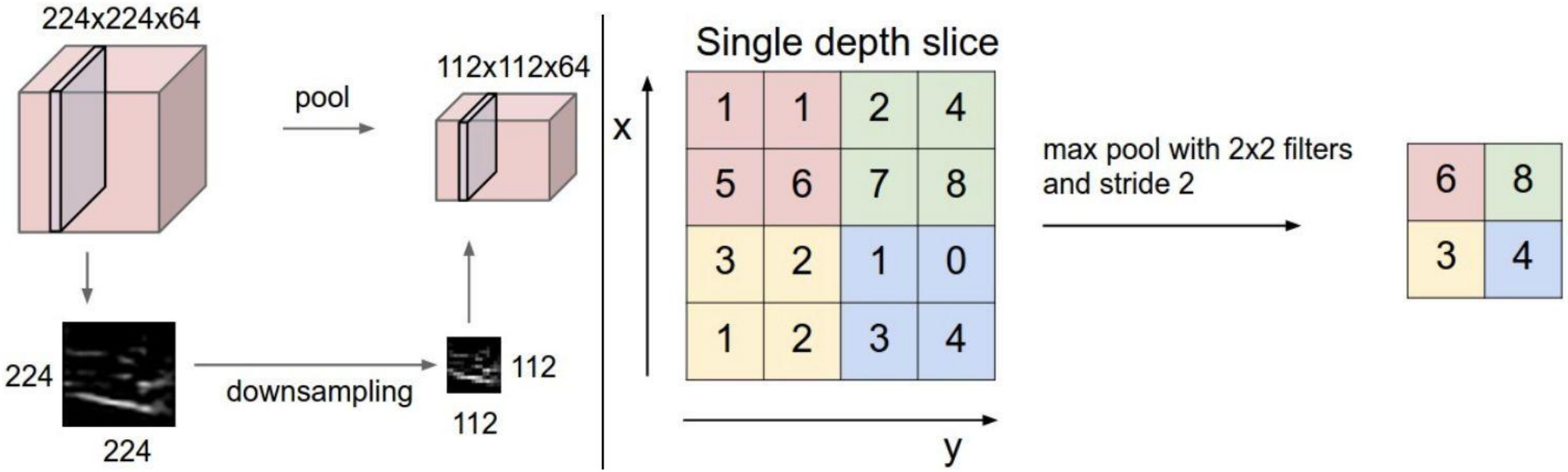


Kernels



Example filters learned by Krizhevsky et al. Each of the 96 filters shown here is of size [11x11x3], and each one is shared by the 55*55 neurons in one depth slice. Notice that the parameter sharing assumption is relatively reasonable: If detecting a horizontal edge is important at some location in the image, it should intuitively be useful at some other location as well due to the translationally-invariant structure of images. There is therefore no need to relearn to detect a horizontal edge at every one of the 55*55 distinct locations in the Conv layer output volume.

Pooling



Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size [224x224x64] is pooled with filter size 2, stride 2 into output volume of size [112x112x64]. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2x2 square).

Pooling layers

- Intuition: the exact location of a feature isn't as important as its rough location
  - Helps prevent overfitting
- Reduces the number of parameters needed in later layers
- L_2 pooling is also common (L_2 norm)

Fully connected layer to combine

- Convolutional layers detected features
- Pooling layers reduced complexity
- Now we have a set of feature maps

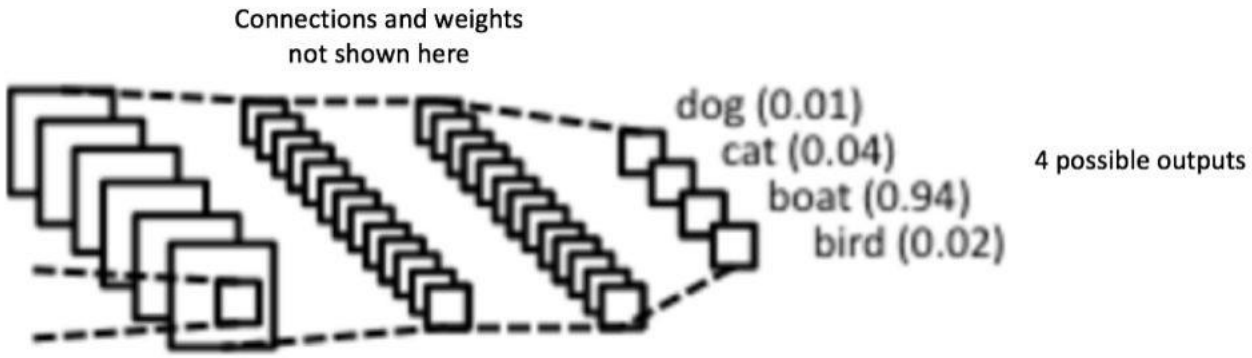
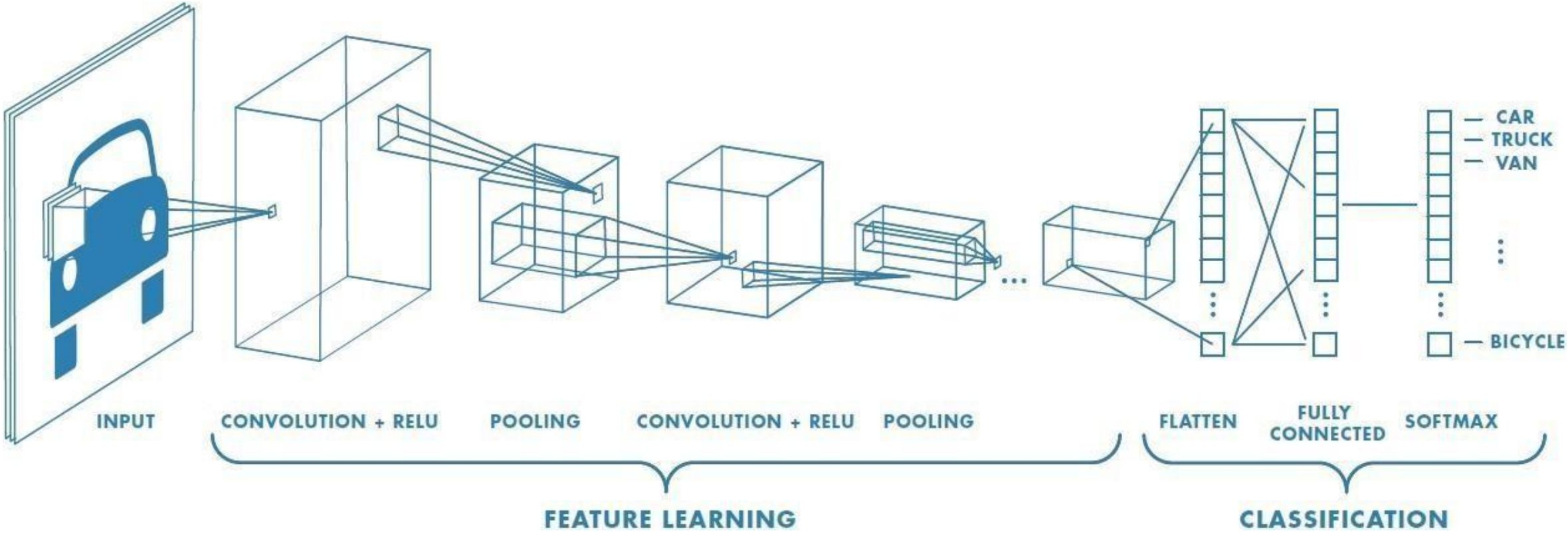


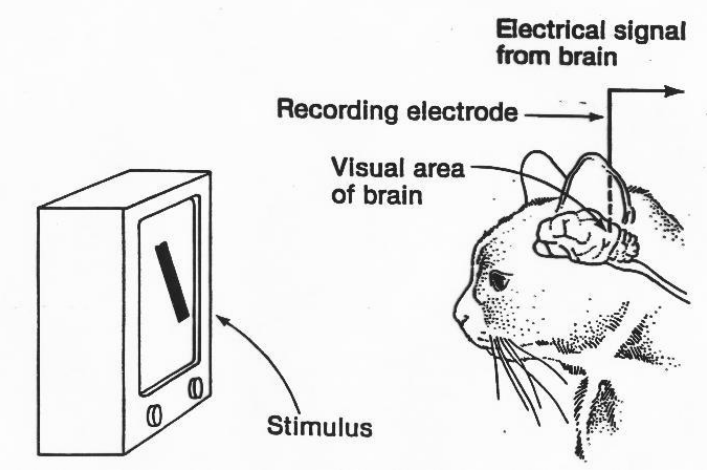
Image Classification with CNN



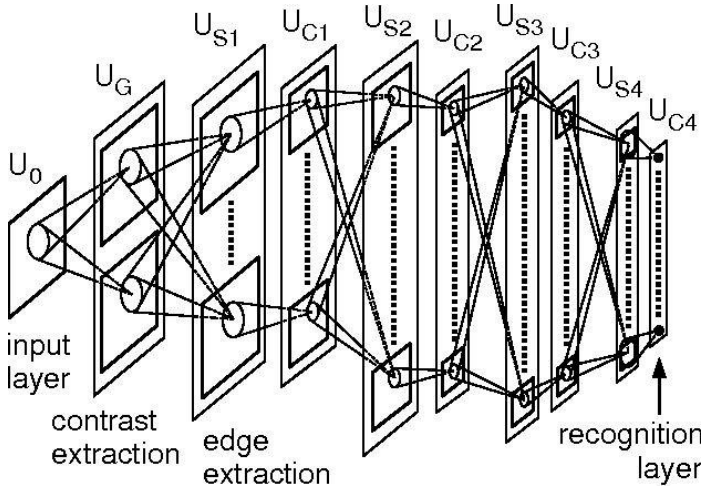
$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as probability of image belonging to a particular class

CNN and brain architecture



Hubel and Wiesel, 1959-1968

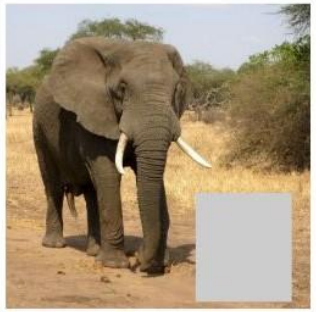


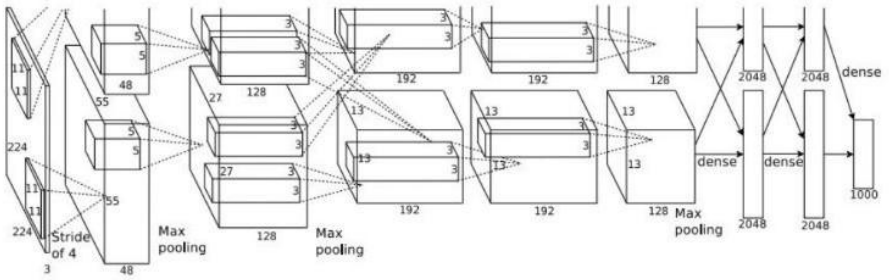
Fukushima, 1980

Brain “inspired” model

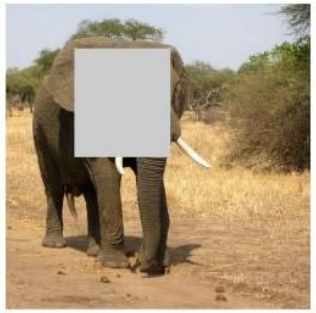
Which pixels matter: Saliency via Occlusion

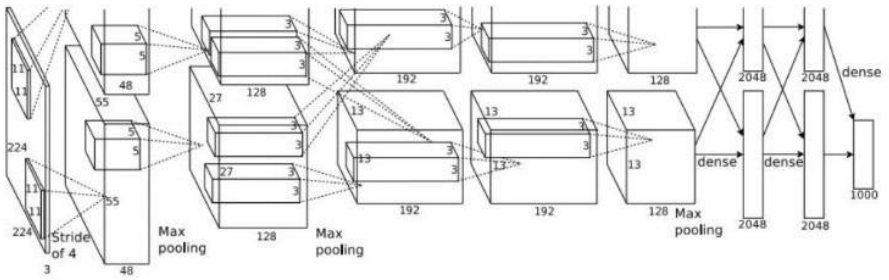
Mask part of the image before feeding to CNN, check how much predicted probabilities change





P(elephant) = 0.95





P(elephant) = 0.75

Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

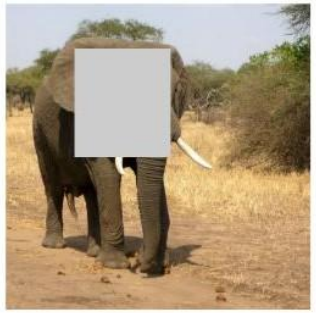
Boat image is CC0 public domain
Elephant image is CC0 public domain
Go-Karts image is CC0 public domain

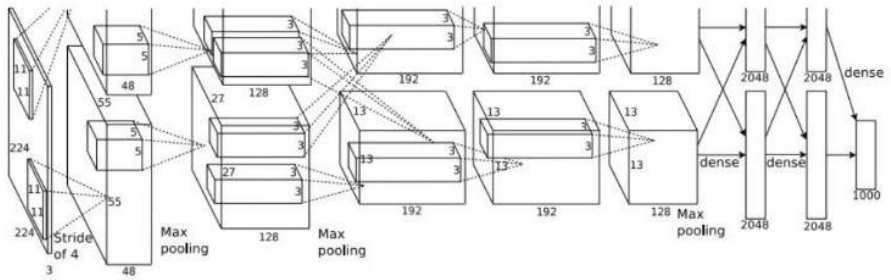
Slide from Fei-Fei Li, Standford lecture

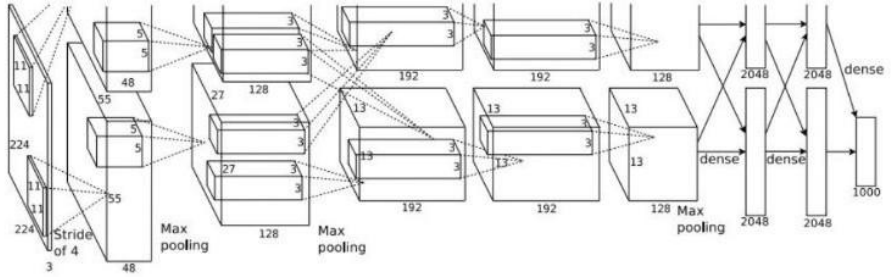
Which pixels matter: Saliency via Occlusion

Mask part of the image before feeding to CNN, check how much predicted probabilities change



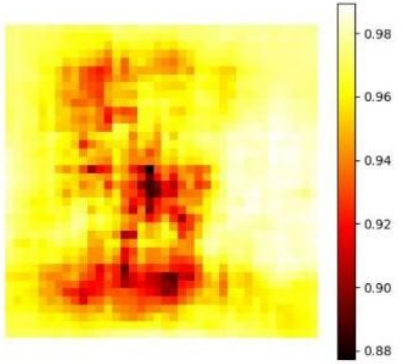






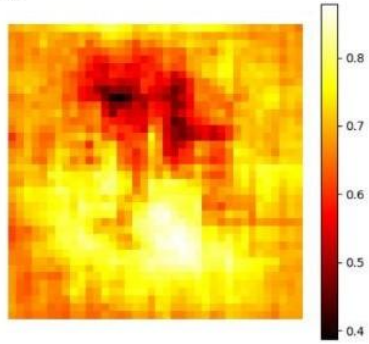
schooner





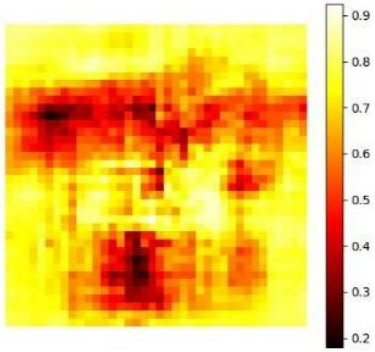
African elephant, Loxodonta africana





go-kart





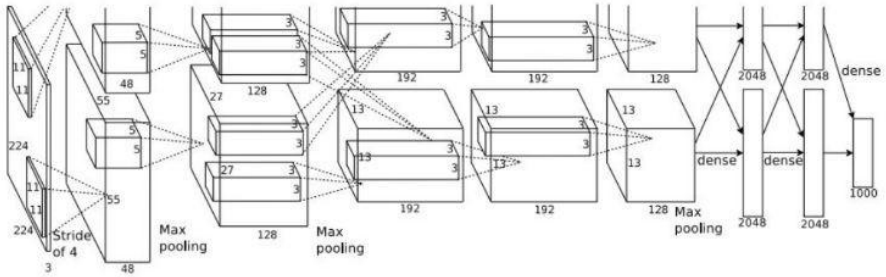
Zeiler and Fergus, “Visualizing and Understanding Convolutional Networks”, ECCV 2014

Boat image is CC0 public domain
Elephant image is CC0 public domain
Go-Karts image is CC0 public domain

Which pixels matter: Saliency via Backprop

Forward pass: Compute probabilities





Dog

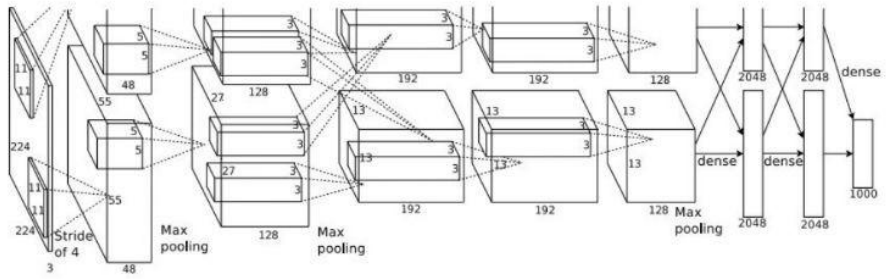
Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR Workshop 2014.

Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

Which pixels matter: Saliency via Backprop

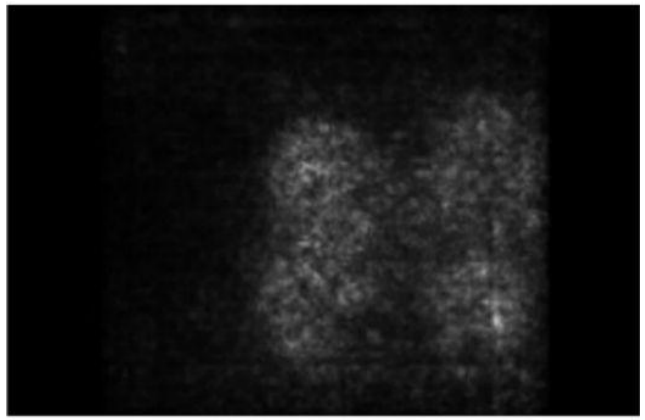
Forward pass: Compute probabilities





Dog

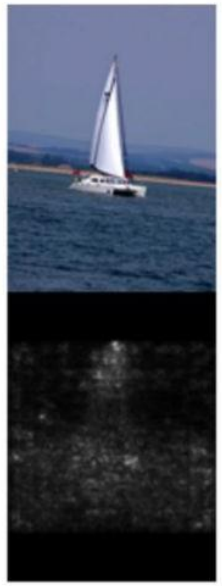
Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels

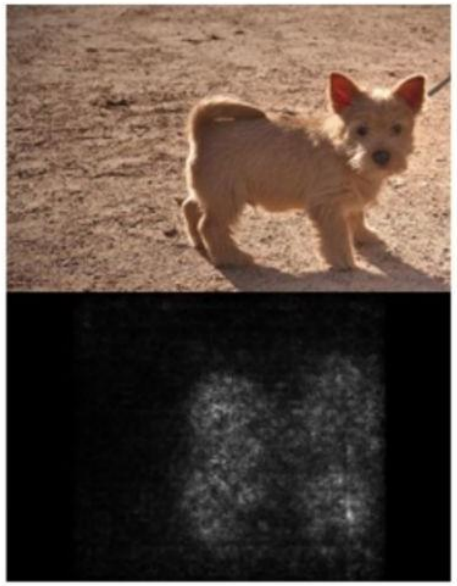


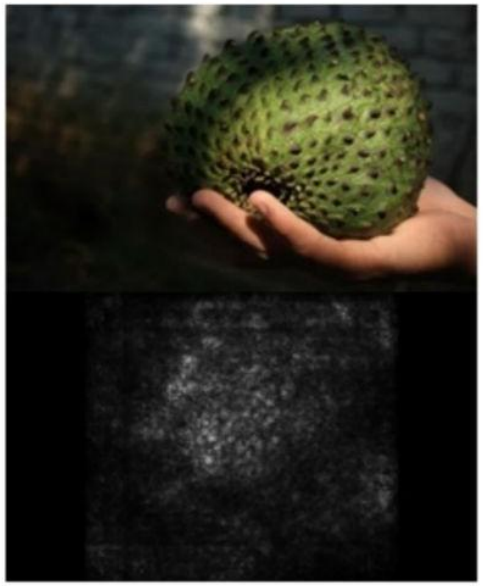
Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR Workshop 2014.

Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

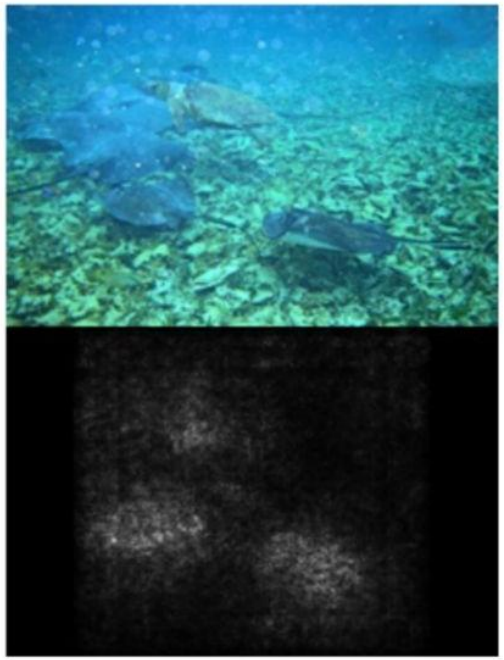
Saliency Maps











Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR Workshop 2014.

Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.